ARENA
ELEMENTAL

Guía de estudio y defensa del proyecto
Simulador de combate por turnos y relevos en C# (Windows Forms)

Fuego **Agua** **Tierra** **Aire**



C# 4.0 · .NET Framework 4.0 · SharpDevelop 4.4

POO · Estructuras de datos · Cálculo 1 · Antropología Filosófica · Redes

Contenido

1 Qué es el proyecto (para decirlo en 1 minuto)

2 Requisitos y cómo ejecutarlo

3 Mapa del proyecto (qué hace cada carpeta y cada archivo)

4 Los datos del juego

5 El backend explicado (lo más importante de defender)

6 ¿Qué pasa cuando haces clic en «Atacar»? (recorrido completo)

7 Conceptos de programación: dónde está cada uno

8 El Terremoto y Cálculo 1

9 Antropología Filosófica: el Códice Elemental

10 El modo LAN (dos PCs)

11 La interfaz explicada de forma sencilla

12 Preguntas que puede hacer el profesor (con respuesta)

13 Ejercicios para practicar (cambios en vivo)

14 Cómo se verificó el proyecto

15 Sobre cómo se hizo el proyecto (transparencia)

16 Qué es básico y qué es avanzado (para saber cuánto profundizar)

17 Glosario

★ Hoja de repaso (1 página)

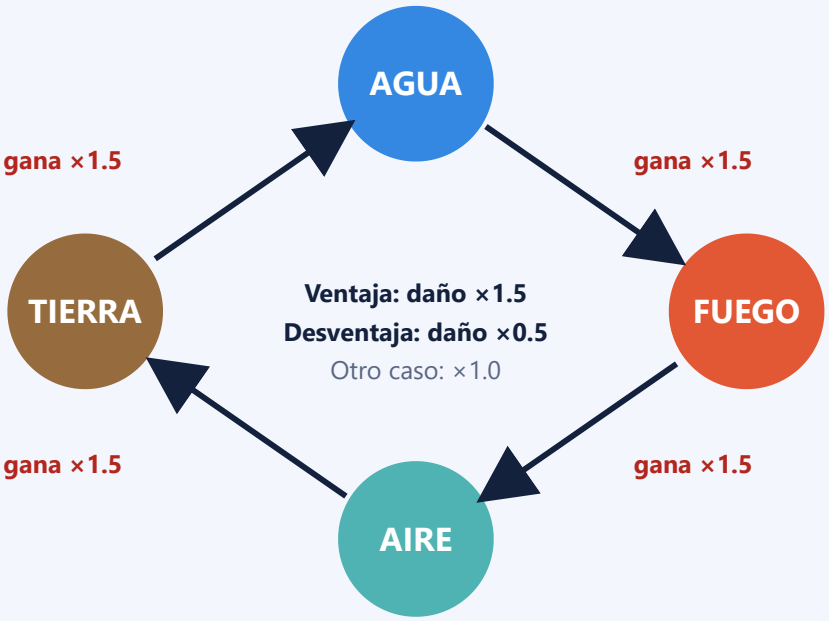
Guía para estudiar, entender y explicar el proyecto. Está pensada para leerse en orden, pero cada sección se puede consultar sola.
Cuando dice «abre el archivo X» es para que lo veas en pantalla mientras lees: **defender un proyecto es poder señalar el código y explicarlo**.

Cómo usar esta guía

1. Lee las secciones 1 a 4 para tener la idea general.
2. Lee la sección 5 con el código abierto (es el corazón del proyecto).
3. Practica la sección 12 (preguntas del profesor) en voz alta.
4. Haz los ejercicios de la sección 13: si puedes cambiar el código en vivo, demuestras que lo entiendes.

1. Qué es el proyecto (para decirlo en 1 minuto)

El triángulo elemental: quién le gana a quién





Pantalla de inicio: nueva partida contra la CPU, partida LAN, estadísticas y el Códice.

Arena Elemental es un juego de combate por turnos hecho en **C# con Windows Forms** (.NET Framework 4.0, SharpDevelop 4.4).

- Cada jugador arma un **equipo de 3 personajes** de 4 elementos: **Fuego, Agua, Tierra y Aire**.
- Los elementos forman un **triángulo de ventajas**: Agua > Fuego > Aire > Tierra > Agua. El que tiene ventaja hace 1.5 veces el daño y el que tiene desventaja, 0.5.
- Solo **un personaje por equipo está en el campo**. Los otros dos esperan en una **cola** (`Queue`) y entran de **relevo** en orden.
- En su turno cada jugador hace **una sola acción**: *Atacar*, usar una de las **2 habilidades** (una de ataque y una de utilidad), *Relevar* o *Pasar* (recupera energía).
- Hay un evento aleatorio, el **Terremoto**, cuyo daño sale de una **derivada** (Cálculo 1).
- Se puede jugar **contra la CPU** o **contra otra persona en otro PC** (modo **LAN** con sockets).
- Hay un **Códice Elemental** con la historia de cada personaje y una reflexión de **Antropología Filosófica**.

Frase para abrir la defensa:

> «Hice un juego de combate por turnos donde apliqué programación orientada a objetos, estructuras de datos (listas, colas y pilas), eventos de Windows Forms, redes con sockets, y lo conecté con Cálculo 1 y Antropología Filosófica.»

2. Requisitos y cómo ejecutarlo

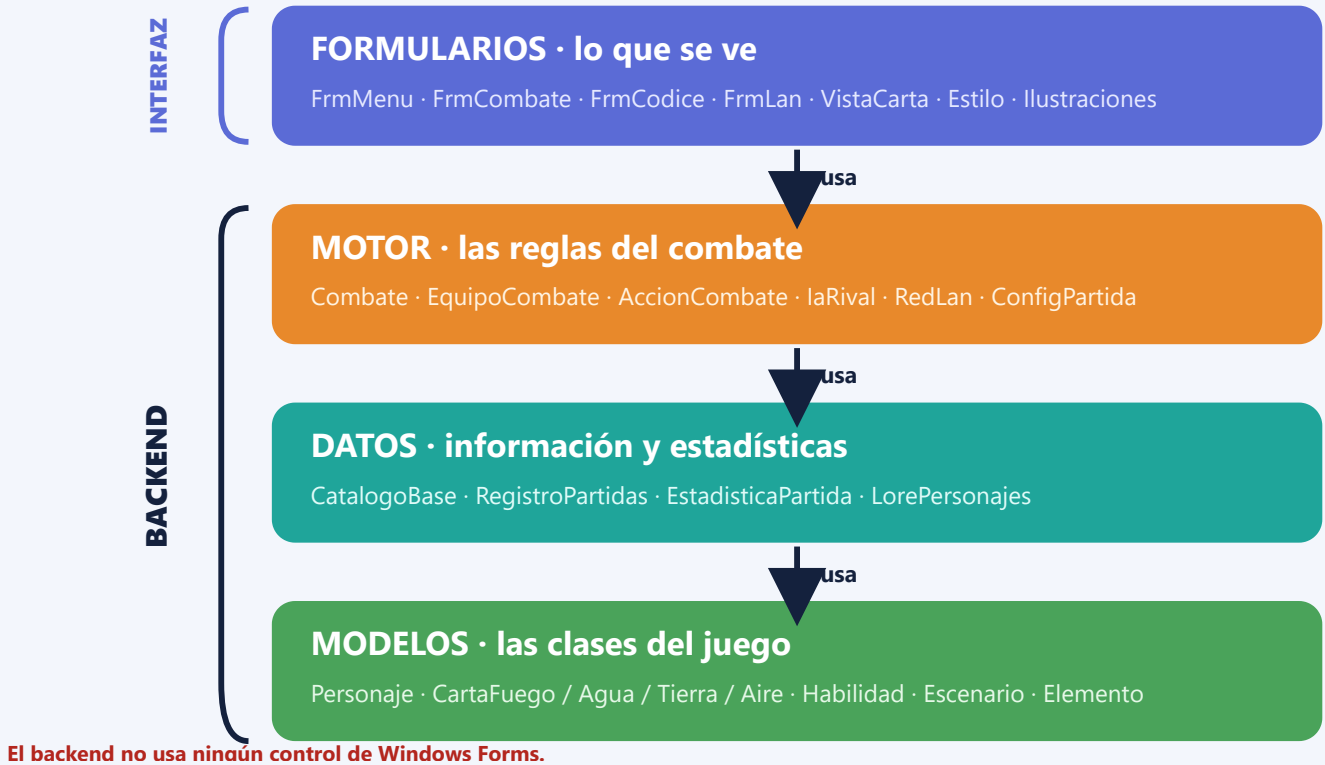
- **Abrir:** `ArenaElemental.sln` con SharpDevelop 4.4 y presionar **F5**.
- **Versión:** .NET Framework 4.0 y **C# 4.0** (por eso no hay `$"..."`, `?.`, `async` ni `=>`).
- **Plataforma:** x86.

- **No usa base de datos**: todo vive en memoria (listas estáticas). Los scripts SQL de la carpeta `BaseDatos` quedaron guardados para una mejora futura.
- **Carpetas que deben estar junto al `.exe`** (o en la carpeta del proyecto): `Imagenes\` (ilustraciones y fondos) y `Sonidos\` (los `.wav`). Si falta algo, el juego **no se cae**: usa colores lisos y sigue sin sonido.

Si Visual Studio no tiene el .NET 4.0, en `ArenaElemental.csproj` se puede cambiar `TargetFrameworkVersion` a `v4.8` sin tocar el código.

3. Mapa del proyecto (qué hace cada carpeta y cada archivo)

Las capas del proyecto (cada una solo conoce a las de abajo)



El namespace principal es `ArenaElemental` y está dividido en capas. **Una regla de oro: las capas de abajo no conocen a las de arriba.**

El **backend** (lo que no es ventana) son las carpetas `Modelos`, `Motor` y `Datos`. **No usa ningún control de Windows Forms**: se podría usar en una consola.

3.1 `Modelos\` — los objetos del juego

Archivo	Qué contiene
`Elemento.cs`	`enum Elemento { Fuego, Agua, Tierra, Aire }`
`Habilidad.cs`	`enum EfectoHabilidad` (los 10 efectos) y la clase `Habilidad`
`Personaje.cs`	Clase **abstracta** `Personaje`: estadísticas, daño, habilidades, turnos
`CartaFuego.cs`, `CartaAgua.cs`, `CartaTierra.cs`, `CartaAire.cs`	Las 4 clases **hijas** de `Personaje`
`Escenario.cs`	Un escenario y su bonus de ataque

3.2 `Motor\` — las reglas del combate

Archivo	Qué contiene
`AccionCombate.cs`	`enum TipoAccion`, `enum EstadoCombate` y la clase `AccionCombate` (la jugada elegida)
`EquipoCombate.cs`	Un equipo de 3: el activo, la **cola** de relevos (`Queue`) y el daño causado
`Combate.cs`	El motor: turnos, acciones, derrotas, Terremoto, muerte súbita y la **pila** del historial (`Stack`)
`IaRival.cs`	Cómo decide la CPU
`RedLan.cs`	Conexión entre dos PCs (sockets `TcpListener` y `TcpClient`)
`ConfigPartida.cs`	El mensaje que se envían los dos jugadores antes de empezar

3.3 `Datos\` — la información fija y las estadísticas

Archivo	Qué contiene
`CatalogoBase.cs`	Las listas estáticas `List<Personaje>` (12) y `List<Escenario>` (4)
`RegistroPartidas.cs`	Lista estática con las partidas jugadas (se pierde al cerrar)
`EstadisticaPartida.cs`	`EstadisticaPartida` (una fila) y `ResumenJugador` (totales)
`LorePersonajes.cs`	Historia, reflexión filosófica y dilema de cada personaje

3.4 `Formularios\` — las ventanas (la interfaz)

Archivo	Qué es
`FrmMenu.cs`	Pantalla de inicio: nombre, botones, héroes de la portada
`FrmSeleccionEquipo.cs`	Elegir los 3 personajes, el nombre del equipo y el escenario
`FrmCombate.cs`	La pantalla de combate (la más grande)
`FrmEstadisticas.cs`	Tabla con las partidas del jugador
`FrmCodice.cs`	Códice: historia + reflexión antropológica + ficha técnica
`FrmLan.cs`, `FrmEsperaLan.cs`	Conexión LAN y ventanita de espera
`VistaCarta.cs`	Control propio que dibuja un personaje como una carta
`MiniaturaPersonaje.cs`	Ficha pequeña de la galería de selección
`AvisoEvento.cs`	El banner de «¡TERREMOTO!»
`FormularioModerno.cs`	Formulario base con el fondo bonito (todas las ventanas heredan de él)
`Estilo.cs`	Colores y funciones de ayuda (crear botones, etiquetas, esquinas redondeadas)
`Ilustraciones.cs`	Prepara las imágenes: quita el fondo blanco y arma las cartas
`Sonidos.cs`	Reproduce los efectos de sonido

Otros: `Program.cs` (punto de entrada: abre `FrmMenu`), `Imagenes\`, `Sonidos\`, `Herramientas\` (código aparte que generó el tablero y los sonidos; **no** forma parte del juego), `BaseDatos\` (scripts SQL guardados).

Nota honesta sobre las ventanas: las ventanas están creadas **por código** (no hay archivos `.Designer.cs`). Funciona igual y se compila igual, pero no se pueden editar arrastrando controles. Si el profesor pregunta por qué: para tener control total del diseño y para que todo el proyecto se pueda leer en archivos de texto.

4. Los datos del juego

4.1 Los 12 personajes

Los cuatro elementos y su papel en el juego

FUEGO · daño puro
Mucho ataque y poca vida. Su habilidad de utilidad sube el ataque. (Pyra, Ignar, Cenizo)

AGUA · apoyo
Equilibrada. Su habilidad de utilidad cura (a uno o a todo el equipo). (Marina, Tidal, Cascada)

TIERRA · tanques
Mucha vida y defensa. Su utilidad sube la defensa o devuelve daño. (Gaia, Gólem, Sílex)

AIRE · veloces
Más energía. Su utilidad da energía, sube el ataque o cura. (Céfiro, Huracán, Brisa)

Valores: Vida (HP), Ataque (ATK), Defensa (DEF), Energía máxima (ENG), Empatía (EMP).

Elemento	Personaje	HP	ATK	DEF	ENG	EMP	Habilidad de ataque	Habilidad de utilidad
Fuego	Pyra	90	18	4	10	3	Llamarada	Aura Ígnea
Fuego	Ignar	100	16	6	10	4	Tajo Carmesí	Grito de Guerra
Fuego	Cenizo	80	20	3	10	2	Explosión de Brasas	Combustión Interna
Agua	Marina	100	13	5	10	8	Chorro a Presión	Marea Curativa
Agua	Tidal	110	12	6	10	7	Oleada Voraz	Oleaje Reparador
Agua	Cascada	95	14	5	10	9	Torrente	Lluvia Sanadora
Tierra	Gaia	125	12	9	8	6	Raíces Punzantes	Muro de Roca
Tierra	Gólem	135	11	11	8	5	Puño Sísmico	Coraza Espinosa
Tierra	Sílex	120	13	8	8	4	Esquirla Cortante	Filo Reflectante
Aire	Céfiro	85	15	4	12	4	Ráfaga Veloz	Viento a Favor
Aire	Huracán	80	17	3	12	3	Tornado Cortante	Ojo de la Tormenta
Aire	Brisa	88	14	5	12	6	Cuchilla de Viento	Soplo Sanador

Idea de diseño (la puedes decir como tu criterio):

- **Fuego** = daño puro (mucho ataque, poca vida). Su utilidad **sube el ataque**.
- **Agua** = apoyo. Su utilidad **cura**.
- **Tierra** = tanques (mucha vida y defensa). Su utilidad **sube la defensa o devuelve daño**.
- **Aire** = veloces (más energía). Su utilidad da **energía, sube el ataque o cura**.

4.2 Los 10 efectos de habilidad

Todas las habilidades se crean con el **mismo constructor**:

```
new Habilidad(nombre, efecto, costo, recarga, potencia, extra, duracion)
```

Efecto	Tipo	`potencia`	`extra`	`duracion`	Qué hace
`GolpePotenciado`	**Ataque**	factor de daño	defensa que ignora	–	Un golpe fuerte
`DobleGolpe`	**Ataque**	factor de cada golpe	–	energía que recupera	Dos golpes
`GolpeVampirico`	**Ataque**	factor de daño	fracción del daño que cura	–	Golpe que cura a quien lo usa
`GolpeConRetroceso`	**Ataque**	factor de daño	fracción de su vida que pierde	–	Golpe muy fuerte que lo lastima (sin matarlo)
`Curar`	**Utilidad**	fracción de vida	–	–	Cura a un aliado
`CurarEquipo`	**Utilidad**	fracción de vida	–	–	Cura a todo el equipo
`PotenciarAtaque`	**Utilidad**	aumento (0.3 = +30 %)	–	turnos	Sube el ataque
`PotenciarDefensa`	**Utilidad**	–	–	turnos	Sube la defensa
`Reflejo`	**Utilidad**	fracción que devuelve	–	turnos	Devuelve parte del daño recibido
`RecargarEnergia`	**Utilidad**	–	–	energía	Recupera energía

Ejemplo: `new Habilidad("Llamarada", EfectoHabilidad.GolpePotenciado, 4, 3, 2.0, 0.5, 0)` = Llamarada, cuesta 4 de energía, recarga 3 turnos, hace daño ×2.0 e ignora el 50 % de la defensa.

4.3 Los 4 escenarios

Volcán Ardiente (Fuego), Lago Cristalino (Agua), Cañón de Roca (Tierra) y Cumbre Ventosa (Aire). Cada uno da **+10 % de ataque** a los personajes de su elemento y cambia la **imagen de fondo** del combate.

4.4 Reglas del juego

1. Equipos de **3 personajes** distintos. El primero sale al campo; los otros esperan en la cola en el orden elegido.
2. Cada turno, **una acción**: Atacar, Habilidad de ataque, Habilidad de utilidad, Relevo o Pasar (+3 de energía).
3. Al empezar su turno, el personaje activo **recupera 2 de energía**, las recargas bajan 1 y los efectos temporales avanzan.
4. **Relevo:** sale el activo, entra el **primero de la cola** (no se elige) y el que salió se forma **al final**. **No se puede relevar dos turnos seguidos.**
5. Si el personaje activo **cae**, entra automáticamente el siguiente de la cola (gratis). El derrotado no vuelve.
6. **Gana** el que deja sin vida a los 3 personajes rivales.
7. **Terremoto:** al empezar cada ronda hay **25 %** de probabilidad. Detalle en la sección 8.
8. **Muerte súbita:** después de la ronda 30, el ataque de todos sube **15 % por ronda**, para que ningún combate sea eterno.
9. **Daño:** `max(1, ATK × (1 + potenciador) × factor × multiplicador elemental × escenario − DEF × (1 − defensa ignorada))`.

Ejemplo con números (el que sale en el historial del juego): Pyra (ATK 18) ataca a Céfiro (Aire, DEF 4) en el Volcán Ardiente (+10 % al Fuego):

```
18 × 1.0 (potenciador) × 1.0 (ataque básico) × 1.5 (Fuego > Aire) × 1.1 (escenario) = 29.7
29.7 − 4 (defensa de Céfiro) = 25.7  → se redondea a 26
```

El historial dice: «Pyra ataca a Céfiro y causa 26 de daño (¡muy efectivo!)».

Ejemplo visual del daño: Pyra ataca a Céfiro en el Volcán Ardiente



5. El backend explicado (lo más importante de defender)

5.1 `Personaje.cs` — la clase base

```
public abstract class Personaje
```

- **Es abstracta**: no se puede hacer `new Personaje(...)`. Solo se crean `CartaFuego`, `CartaAgua`, `CartaTierra` o `CartaAire`.
- **Encapsulamiento:** los datos importantes son **campos privados** (`private int hp;`) y se acceden con **propiedades**. Por ejemplo `Hp`:

```
public int Hp
{
    get { return hp; }
    set
    {
        if (value < 0) value = 0;
        if (value > HpMax) value = HpMax;
        hp = value;
    }
}
```

Así la vida **nunca** queda negativa ni sobre el máximo, sin importar quién la cambie. Lo mismo pasa con `Energia`.

- **Propiedades con `private set`:** `Nombre`, `HpMax`, `Atk`, etc. se pueden **leer** desde fuera pero solo se **asignan** dentro de la clase (en el constructor).
- **Constructor `protected`:** solo lo pueden llamar las clases hijas. Valida que no haya nombre vacío, estadísticas negativas ni habilidades en el orden equivocado (si pasa, lanza una excepción).
- **Constantes:** `MultiplicadorVentaja = 1.5`, `MultiplicadorDesventaja = 0.5`, `EnergiaPorTurno = 2` …

Métodos más importantes:

Método	Qué hace
`CalcularDano(objetivo, factor, defensaIgnorada)`	La fórmula del daño (privado)
`Atacar(objetivo)`	Ataque básico. Devuelve el **texto** que va al historial
`RecibirDano(dano)`	Resta vida y devuelve cuánta perdió realmente
`Curar(cantidad)`	Suma vida (sin pasar del máximo y sin revivir)
`RecuperarEnergia(cantidad)`	Suma energía
`PuedeUsarHabilidad(numero)`	Vivo + sin recarga + energía suficiente
`UsarHabilidad(numero, objetivo, equipo)`	Usa la habilidad 0 o 1
`IniciarTurno()`	Baja las recargas, da 2 de energía y descuenta los turnos de los efectos
`AlEntrarEnRelevo()` / `AlSalirDelCampo()`	Energía por empatía al entrar; limpia los efectos al salir

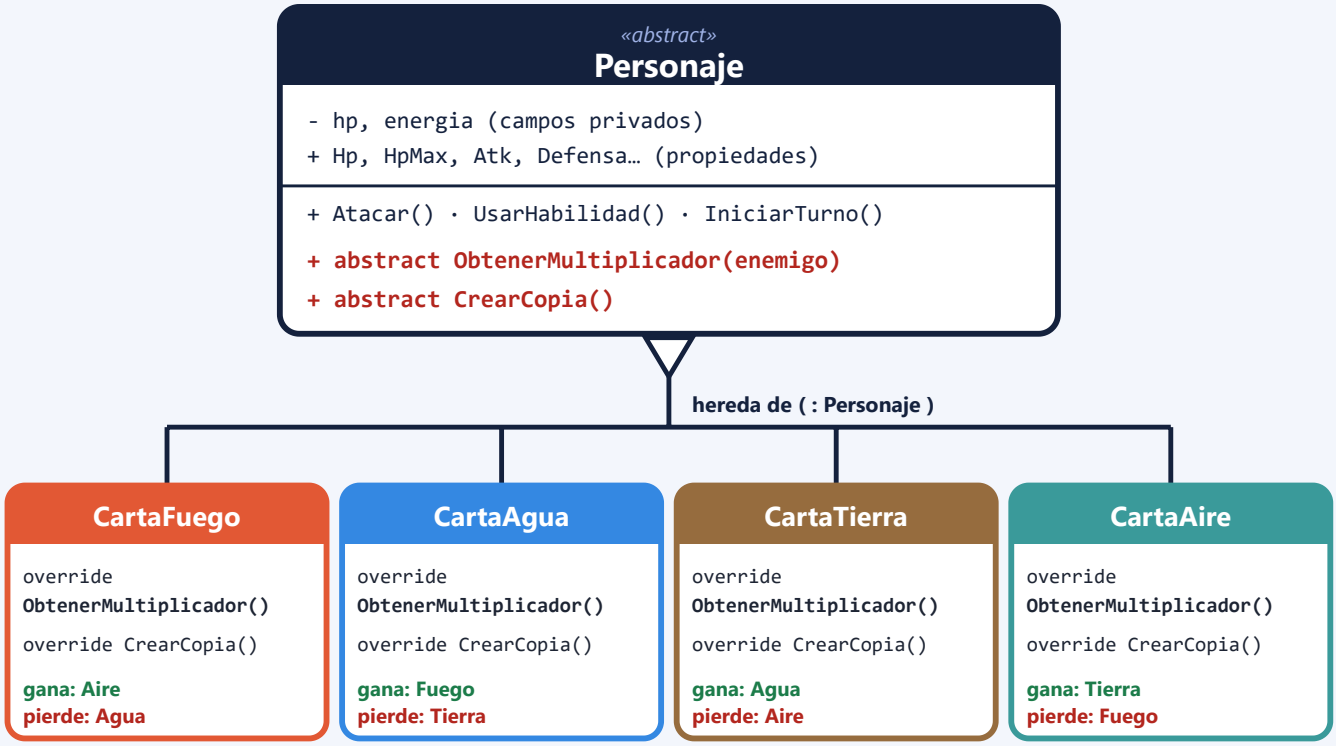
Los efectos temporales (potenciador de ataque, defensa extra, reflejo) son campos privados: `bonoAtaque`, `turnosBonoAtaque`, `bonoDefensa`, `turnosBonoDefensa`, `reflejo`, `turnosReflejo`. Un efecto con «2 turnos» se descuenta en `IniciarTurno()` y cuando llega a 0 se apaga.

Detalle que suelen preguntar: el potenciador de ataque guarda `turnosBonoAtaque = Duracion + 1` porque el contador baja justo al empezar el siguiente turno; sin ese +1 el efecto duraría un turno menos de lo que dice la habilidad.

Reglas de seguridad (para que nadie muera «por accidente»): el **retroceso**, el **reflejo** y el **Terremoto** nunca dejan a alguien en menos de 1 de vida.

5.2 Herencia y polimorfismo: `CartaFuego`, `CartaAgua`, `CartaTierra`, `CartaAire`

Herencia y polimorfismo (diagrama de clases)



Polimorfismo: el motor llama `activo.ObtenerMultiplicador(...)` sin saber el elemento; cada objeto responde a su manera.

Son 4 clases casi idénticas (`: Personaje`). Lo único propio de cada una son **dos métodos que sobrescriben (`override`) los de la clase base**:

```
public override double ObtenerMultiplicador(Elemento enemigo)   // el triángulo
public override Personaje CrearCopia()                           // cómo copiarse
```

Ejemplo, `CartaFuego`:

```
if (enemigo == Elemento.Aire) return MultiplicadorVentaja;     // Fuego gana contra Aire
if (enemigo == Elemento.Agua) return MultiplicadorDesventaja;  // Fuego pierde contra Agua
return MultiplicadorNeutro;
```

Esto es polimorfismo: el motor llama `activo.ObtenerMultiplicador(...)` sin saber si es de Fuego, Agua, Tierra o Aire; cada objeto responde a su manera. En `Combate` y en `IaRival` casi nunca verás un `if (elemento == ...)`.

¿Para qué sirve `CrearCopia()`? El catálogo (`CatalogoBase`) guarda los personajes «originales». Al armar un equipo se usan **copias** (`EquipoCombate` llama a `CrearCopia()`), así el combate puede bajar la vida, gastar energía, etc. sin dañar los originales y se pueden jugar muchas partidas seguidas.

5.3 `Habilidad.cs`

Una clase con un solo constructor y los siete datos que describen el efecto. Tiene:

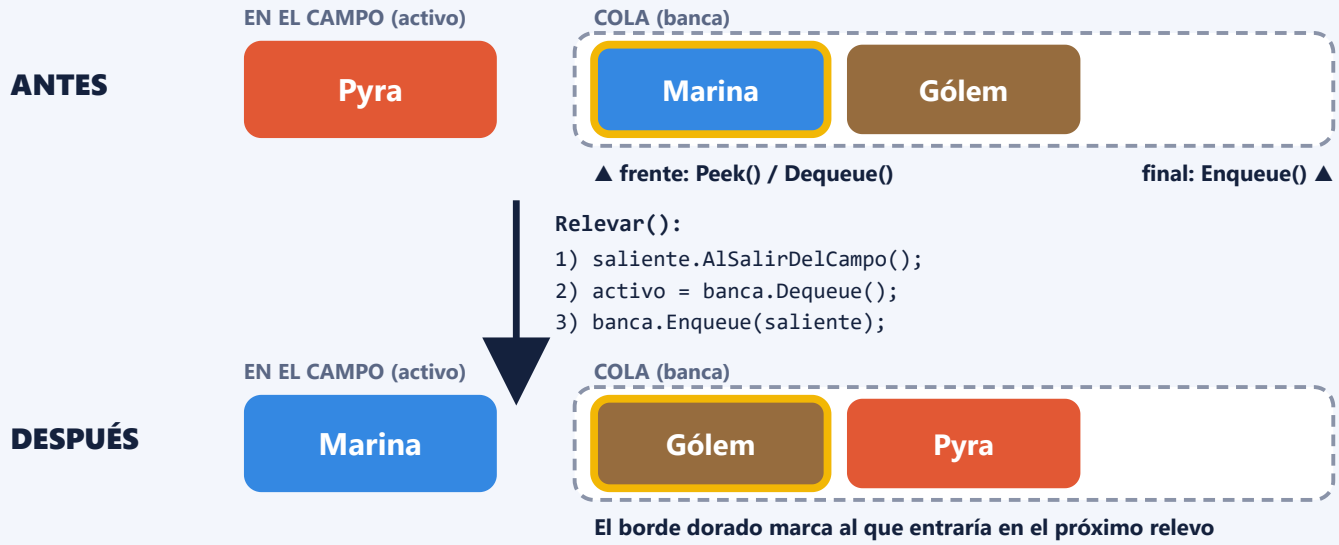
- `EsOfensiva`: true para los 4 efectos de ataque.
- `RequiereAliado`: true solo para `Curar` (el jugador elige a quién).
- `CooldownActual` (turnos que faltan para poder usarla otra vez), `Usar()` (empieza la recarga y suma un uso) y `PasarTurno()` (baja la recarga 1).
- `Clonar()`: una copia nueva sin recarga.
- `Descripcion`: arma el texto que se ve en los tooltips; usa `CultureInfo.InvariantCulture` para que el decimal salga con punto (`2.0`) en cualquier computadora.

5.4 `Escenario.cs`

Guarda nombre, elemento afín y bonus. `MultiplicadorPara(elemento)` devuelve `1.1` si el elemento es el afín y `1.0` si no; `AplicarA(personaje)` se lo asigna al personaje. `Combate` lo llama al empezar la partida.

5.5 `EquipoCombate.cs` — la COLA (`Queue`)

La cola de relevos (Queue): primero en entrar, primero en salir



Si el que sale está derrotado **no** vuelve a la cola. Tampoco se puede relevar dos turnos seguidos (`RelevoBloqueado`).

```
private List<Personaje> miembros = new List<Personaje>();   // los 3, para mostrarlos
private Queue<Personaje> banca = new Queue<Personaje>();    // la cola de relevos
private Personaje activo;
```

- En el **constructor**: se hacen copias, `activo = miembros[0]` y los otros dos se meten con `Enqueue`.
- `Relevar()` es el uso clásico de una cola (FIFO: *primero en entrar, primero en salir*):

```
Personaje saliente = activo;
saliente.AlSalirDelCampo();
activo = banca.Dequeue();            // sale de la cola el que lleva más tiempo esperando
if (saliente.EstaVivo)
    banca.Enqueue(saliente);         // el que salió se forma al final
```

- `ProximoRelevo` usa `Peek()` (mira el primero **sin sacarlo**). La pantalla lo usa para marcar con borde dorado la carta que entraría.
- `RelevoBloqueado` implementa la regla «no relevar dos turnos seguidos».
- Valida que el equipo tenga exactamente 3 personajes **y que no haya repetidos** (si no, lanza `ArgumentException`).

5.6 `AccionCombate.cs`

Representa **la jugada elegida**: un tipo (`TipoAccion`), el número de habilidad (0 o 1) y, si cura a un aliado, la posición de ese aliado. El constructor es **privado** y se usan métodos estáticos que se leen como una frase:

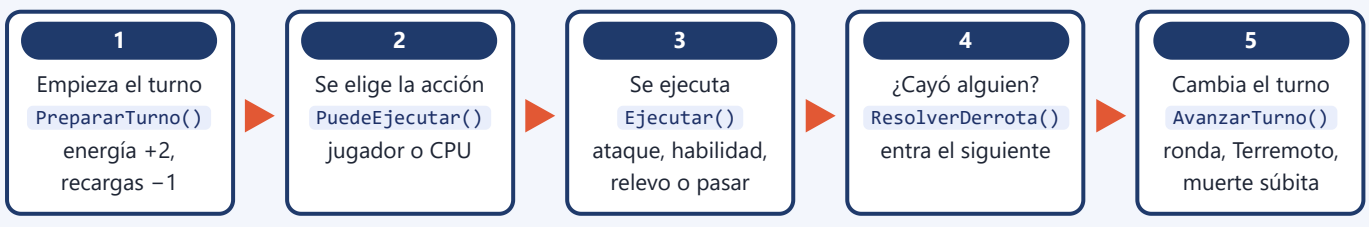
```
AccionCombate.Atacar()
AccionCombate.Habilidad(0)
AccionCombate.HabilidadSobreAliado(1, 2)
AccionCombate.Relevar()
AccionCombate.Pasar()
```

5.7 `Combate.cs` — el motor

El historial del combate (Stack): último en entrar, primero en salir



Ciclo de un turno dentro de Combate



Es la clase central. Guarda: el escenario, los dos equipos, de quién es el turno, el número de ronda, el estado (en curso / victoria) y el **historial** como una **pila** (`Stack<string>`):

```
private Stack<string> historial = new Stack<string>();
private void Registrar(string mensaje) { historial.Push(mensaje); }      // agregar
public string UltimaJugada { get { return historial.Peek(); ... } }      // ver la última
```

Al recorrerla con `foreach` sale **primero lo más reciente** (LIFO: *último en entrar, primero en salir*). Por eso el registro de la pantalla muestra arriba lo último que pasó.

Los dos métodos principales:

1. `PuedeEjecutar(accion, out motivo)` — dice si la acción es válida y, si no, **por qué** (por ejemplo «Energía insuficiente» o «No se puede relevar dos turnos seguidos»). Usa `out` para devolver dos cosas: un `bool` y un texto.
2. `Ejecutar(accion)` — hace la jugada completa:

```
1. Verifica que se pueda (si no, lanza InvalidOperationException)
2. Hace la acción (EjecutarAtaque / EjecutarHabilidad / Relevar / EjecutarPasar)
3. Anota si fue un relevo (para bloquear el siguiente)
4. ResolverDerrota(): si el activo rival cayó, entra el siguiente (o termina el combate)
5. AvanzarTurno(): pasa el turno; si vuelve al Jugador, empieza una ronda nueva
```

`AvanzarTurno()` es donde cada ronda nueva hace: `Turno++` , aplicar muerte súbita (si corresponde), **tirar el dado del Terremoto** y preparar el turno del equipo que sigue (`PrepararTurno()` llama a `IniciarTurno()` del personaje).

Quién empieza: siempre el `Jugador` . En una partida LAN, el anfitrión es el `Jugador` en los dos PCs.

5.8 `IaRival.cs` — la CPU

No es inteligencia artificial real: es una lista de reglas en orden de prioridad (la primera que se cumple, gana):

1. Releva si está en **desventaja** elemental y el que sigue en la cola le iría mejor (55 % de las veces).
2. Releva si le queda **menos del 25 %** de vida (35 %).
3. Usa su **habilidad de utilidad** si sirve en ese momento (curar si a alguien le falta bastante vida, potenciar si no tiene el potenciador, etc.).
4. Usa su **habilidad de ataque** (75 %).
5. **Descansa** (`Pasar`) si le falta energía (30 %).
6. Si nada de lo anterior, **ataca**.

Las probabilidades se calculan con `Probabilidad(porcentaje)` → `azar.Next(100) < porcentaje` . `Decidir()` siempre devuelve una acción **válida**.

5.9 `CatalogoBase.cs`

Dos listas estáticas, `Personajes` y `Escenarios` , que se llenan en un **constructor estático** (se ejecuta una sola vez, la primera vez que se usa la clase). Cualquier ventana puede leer `CatalogoBase.Personajes` sin crear objetos.

5.10 Estadísticas: `RegistroPartidas.cs` y `EstadisticaPartida.cs`



Pantalla de estadísticas: tarjetas de resumen y tabla de partidas (viene de RegistroPartidas).

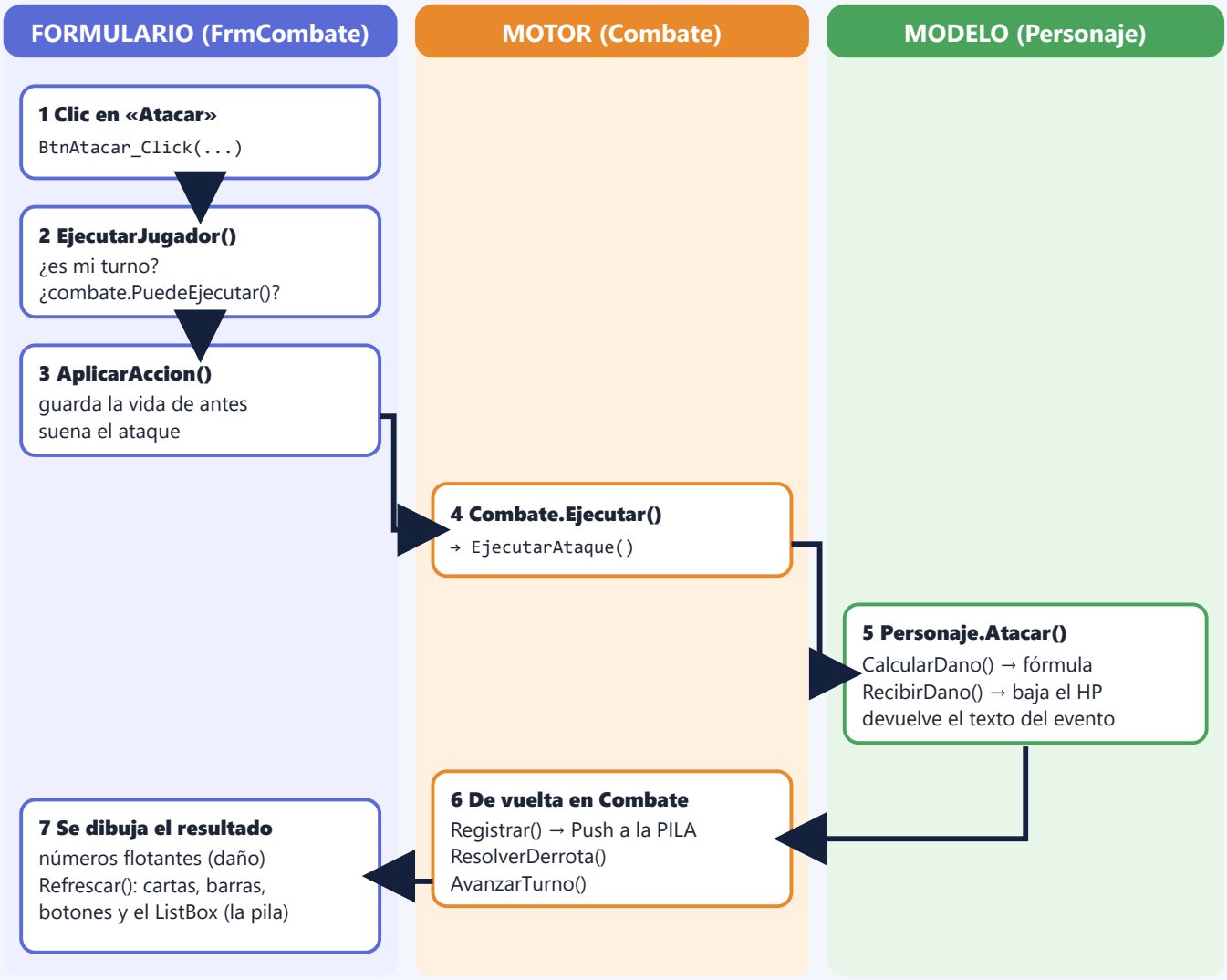
- `RegistroPartidas.Agregar(...)` guarda una partida en una lista estática. `DeJugador(nombre)` devuelve las de ese jugador (de la más nueva a la más vieja).
- `ResumenJugador` recorre esa lista y calcula: partidas, victorias, derrotas, % de victorias, daño total y rondas promedio.
- **Se pierden al cerrar el programa** (no hay base de datos). Esto fue una decisión del proyecto: la BD quedó como mejora futura (scripts en `BaseDatos\`).

5.11 Estructuras de datos usadas (para la rúbrica)

Estructura	Dónde	Para qué
`Queue<Personaje>` (cola)	`EquipoCombate.banca`	Orden de los relevos (FIFO)
`Stack<string>` (pila)	`Combate.historial`	Historial de jugadas (LIFO)
`List<Personaje>`	`CatalogoBase`, `EquipoCombate.miembros`	Catálogo y miembros
`List<EstadisticaPartida>`	`RegistroPartidas`	Historial de partidas
`Dictionary<string, FichaLore>`	`LorePersonajes`	Buscar el lore por nombre
Arreglos (`Button[]`, `VistaCarta[]`)	Formularios	Grupos de controles
`ListBox`	`FrmCombate`	Muestra el contenido de la pila
`FlowLayoutPanel`	`FrmCombate`	Muestra la cola como «mano de cartas»

6. ¿Qué pasa cuando haces clic en «Atacar»? (recorrido completo)

Recorrido de un clic en «Atacar» a través de las tres capas



Paso 8: si ahora le toca a la CPU, un `Timer` espera 0.9 s, llama a `IaRival.Decidir()` y se repite del paso 3 al 7. **La ventana nunca calcula reglas: solo pide al motor y dibuja.**



La pantalla de combate: cartas, bancas (la cola), botones de acción y el historial (la pila).

Este recorrido sirve para explicar **cómo se conectan las capas**. Sigue los nombres en el código:

1. **Ventana** — `FrmCombate.BtnAtacar_Click` llama a `EjecutarJugador(AccionCombate.Atacar())`.
2. `EjecutarJugador` revisa que sea tu turno (`EsMiTurno`) y que `combate.PuedeEjecutar(...)` sea `true`. Si no, muestra el motivo en un `MessageBox`.
3. `AplicarAccion(accion)` anota la vida de los dos personajes en el campo, reproduce el sonido (`Sonidos.Ataque()`) y llama al motor.
4. **Motor** — `Combate.Ejecutar(accion)` → `EjecutarAtaque(...)` → `Personaje.Atacar(enemigo)`.
5. **Modelo** — `Atacar` calcula el daño con `CalcularDano` (fórmula de la sección 4.4), hace `objetivo.RecibirDano(...)`, y devuelve un texto.
6. De vuelta en `Combate`: el texto se guarda en la **pila** (`Registrar`), se revisa si el rival cayó (`ResolverDerrota`) y se pasa el turno (`AvanzarTurno`).
7. De vuelta en `AplicarAccion`: compara la vida de antes y de después para mostrar los **números flotantes** (rojo = daño, verde = cura) y llama a `Refrescar()`.
8. `Refrescar()` actualiza **todo**: las cartas, las bancas, los botones, el texto del turno y el `ListBox` con la pila.
9. `ProcesarSiguiente()`: si el combate terminó, muestra el resultado y guarda la estadística; si ahora le toca a la CPU, arranca un `Timer`, y 0.9 segundos después `TimerCpu_Tick` le pide una jugada a `IaRival.Decidir(...)` y repite el mismo camino con `AplicarAccion`.

Idea clave para decir: *«La ventana nunca calcula nada de las reglas. Solo le pide al motor que ejecute una acción y después dibuja el estado. Por eso las reglas se pueden probar sin ventanas.»*

7. Conceptos de programación: dónde está cada uno

Concepto	Dónde verlo
Clase y objeto	Todas: `Personaje`, `Habilidad`, `Escenario`, `Combate`… Un objeto es, por ejemplo, la Pyra de una partida concreta
Constructor	`Personaje`, `EquipoCombate`, `Habilidad`, `Combate`
Sobrecarga de constructores	`Combate` tiene 2 (con y sin semilla); `FrmCombate` también (contra CPU / LAN)

Encapsulamiento	Campos `private` + propiedades en `Personaje`, `EquipoCombate`, `Combate`
Herencia	`CartaFuego : Personaje` (y las otras tres); `FrmMenu : FormularioModerno : Form`
Clase abstracta	`Personaje` (`abstract`)
Polimorfismo	`ObtenerMultiplicador` y `CrearCopia` (`override`)
Enumeraciones	`Elemento`, `EfectoHabilidad`, `TipoAccion`, `EstadoCombate`
Constantes	`Personaje.MultiplicadorVentaja`, `Combate.ProbabilidadTerremoto` …
Miembros estáticos	`CatalogoBase`, `RegistroPartidas`, `Estilo`, `Sonidos`, `AccionCombate.Atacar()`
Listas, cola, pila, diccionario	Sección 5.11
Excepciones	Validaciones con `throw new ArgumentException(...)`; `try/catch` en `RedLan`, `Sonidos` e `Ilustraciones`
Parámetro `out`	`Combate.PuedeEjecutar(accion, out motivo)`
Eventos de Windows Forms	`boton.Click += BtnAtacar_Click;`
`Timer`	Turno de la CPU, temblor de la ventana, números flotantes, revisión de la red
Cuadros de diálogo	`ShowDialog()` (abre una ventana y espera), `MessageBox.Show(...)`
Control propio (`UserControl`)	`VistaCarta`, `MiniaturaPersonaje`
Sockets	`RedLan`

Cómo funcionan los eventos aquí

En Windows Forms, un **evento** es «algo que pasa» (un clic) y un **manejador** es el método que se ejecuta cuando pasa. En este proyecto siempre se hace igual:

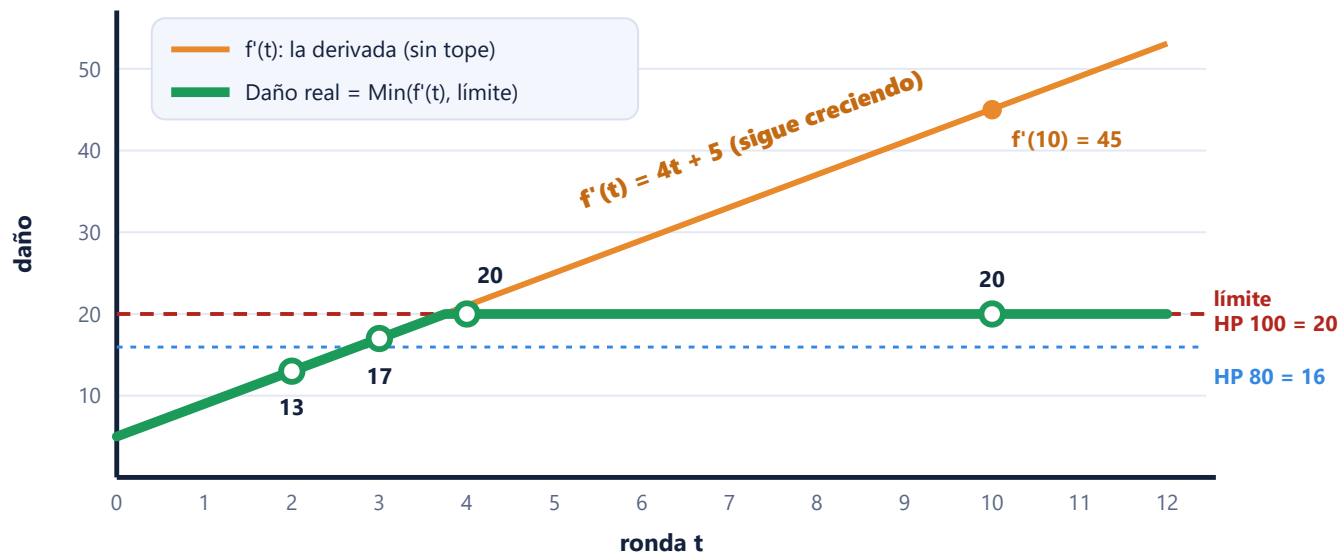
```
btnAtacar.Click += BtnAtacar_Click;              // «cuando hagan clic, ejecuta este método»

private void BtnAtacar_Click(object sender, EventArgs e)
{
    EjecutarJugador(AccionCombate.Atacar());
}
```

Cuando varios controles comparten manejador (por ejemplo, los 4 botones de escenario) se distingue cuál fue con la propiedad `Tag`: al crear el botón se guarda su número (`b.Tag = i;`) y en el manejador se lee: `(int)((Button)sender).Tag`.

8. El Terremoto y Cálculo 1

Daño del Terremoto = Min(f'(t) , límite) con f(t) = 2t² + 5t y f'(t) = 4t + 5



Con 100 de vida el daño crece 13 → 17 → 20 y desde la ronda 4 se queda **constante en 20**: el `Min` «acota» la derivada (el límite). Con 80 de vida el tope es 16.

Qué pasa: al empezar cada ronda hay **25 %** de probabilidad de Terremoto (`azar.Next(100) < 25` en `Combate.AvanzarTurno`). Si ocurre, los **dos personajes que están en el campo** pierden vida. La ventana tiembla, suena un efecto y aparece un banner dorado.

La matemática:

- **Función:** `f(t) = 2t² + 5t`, donde `t` es el número de ronda.
- **Derivada** (regla de la potencia: `d/dt(tⁿ) = n·tⁿ⁻¹`):
  - `d/dt (2t²) = 2·2·t = 4t`
  - `d/dt (5t) = 5`
  - Entonces `f'(t) = 4t + 5`.
- **Daño =** `Min(f'(t), límite)`, con **límite = 20 % de la vida máxima** del personaje.

¿Por qué una derivada? `f(t)` representa la «energía sísmica acumulada» hasta la ronda `t`. Su derivada `f'(t)` es **qué tan rápido crece** esa energía en esa ronda: es lo que «golpea» ahora. Como es lineal, el golpe crece de 4 en 4 cada ronda.

¿Por qué el `Min()`? (el vínculo con límites) Sin tope, `f'(t) = 4t + 5` crece sin parar: en la ronda 30 haría 125 de daño y mataría a todos. Con el `Min()` el daño se «acota»: nunca pasa de un valor fijo. Visto como función de `t`, el daño crece y a partir de cierta ronda se queda **constante** (se «aplana»).

Ejemplos (personaje con 100 de vida → límite 20):

Ronda `t`	`f'(t) = 4t + 5`	Daño = `Min(f'(t), 20)`
2	13	13
3	17	17
4	21	**20** (llega al límite)
10	45	20

Con 80 de vida el límite es 16 (el daño se aplana desde la ronda 3). Se calcula **por personaje** porque cada uno tiene su vida máxima.

Seguridad: además, el Terremoto **no puede dejar a nadie con menos de 1 de vida**, así que no elimina a nadie.

En el código (`Combate.cs`): `FuncionTerremoto` , `DerivadaTerremoto` y `DanoDelTerremoto` son métodos `static` (matemática pura, sin estado); `ProvocarTerremoto()` y `AplicarDanoTerremoto()` aplican el daño. **La fórmula queda escrita en la pila del historial**:

```
¡TERREMOTO! en Cañón de Roca: f(t) = 2t² + 5t, f'(t) = 4t + 5, con t = 35 => f'(35) = 145.
   Sílex: Min(f'(t) = 145, límite = 24) => Daño: 1 (1/120 HP).
```

(En ese ejemplo le quedaba 2 de vida: el daño se redujo a 1 para que no muriera.)

El azar es «sembrado»: `new Random(semilla)` . Con la misma semilla y las mismas jugadas, dos combates dan **exactamente** el mismo Terremoto. Esto es lo que permite sincronizar el modo LAN (sección 10).

9. Antropología Filosófica: el Códice Elemental

Un eje filosófico por elemento

FUEGO
Pasión, voluntad de poder y autodestrucción (finitud)
Nietzsche · Schopenhauer · Freud · Bataille

AGUA
Alteridad, empatía y ética del cuidado
Levinas · Buber · Edith Stein · Gilligan

TIERRA
Trascendencia, orden y resistencia ante la finitud
Heidegger · Spinoza · Unamuno · Arendt

AIRE
Libertad, contingencia y búsqueda de sentido
Sartre · Kierkegaard · Camus · Frankl



Códice Elemental, pestaña «Dimensión Antropológica» (Marina).

Dónde: botón «CÓDICE ELEMENTAL» del menú → `FrmCodice`. Los textos están en `Datos\LorePersonajes.cs`.

Cómo está armado en el código:

- `FichaLore`: una clase con `Titulo`, `Concepto`, `Historia`, `Reflexion` y `Dilema`.
- `LorePersonajes`: un `Dictionary<string, FichaLore>` (la clave es el nombre del personaje) y métodos para consultarlo: `ObtenerFicha`, `EjeDe(elemento)`, `ExplicacionDelEje(elemento)`, `LecturasDe(elemento)`.
- `FrmCodice`: a la izquierda, los 12 personajes por elemento; al centro la carta y el **dilema**; a la derecha tres **pestañas**: *Historia & Lore*, *Dimensión Antropológica* y *Ficha Técnica*.

Cada facción representa un eje antropológico:

Elemento	Eje	Autores que se citan
Fuego	Pasión, **voluntad de poder** y autodestrucción (finitud)	Nietzsche, Schopenhauer, Heráclito, Freud, Bataille
Agua	**Alteridad**, empatía y **ética del cuidado**	Levinas, Buber, Edith Stein, Gilligan, Noddings, Ricoeur
Tierra	**Trascendencia**, orden y resistencia ante la **finitud**	Heidegger, Spinoza, Unamuno, Platón, Arendt, Pico della Mirandola
Aire	**Libertad**, contingencia y búsqueda de **sentido**	Sartre, Kierkegaard, Camus, Frankl, Aristóteles

Cómo se conecta el personaje con la idea (ejemplos para la defensa):

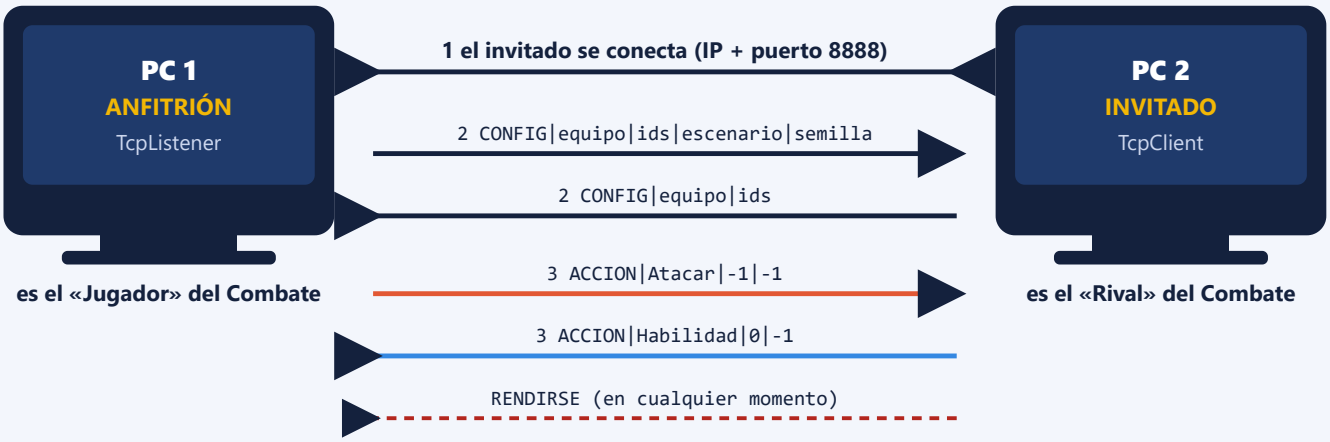
- **Cenizo** (Fuego): su habilidad «Explosión de Brasas» le **devuelve daño** a quien la usa. En el juego, *lo que gana en intensidad lo paga en vida*: es la autodestrucción de la pasión (Freud, Bataille).
- **Marina** (Agua): su habilidad **cura**. Es la responsabilidad ante el rostro del Otro de Levinas: cuidar al que sufre.
- **Tidal** (Agua): su «Oleada Voraz» le roba vida al rival para cuidar a los suyos. Pregunta: ¿se puede cuidar a unos usando a otros como medio? (Kant, Buber).
- **Gaia** (Tierra): «Muro de Roca» sube la defensa. Es la serenidad ante la finitud (Platón: filosofar es «practicar el morir»; Heidegger: ser-para-la-muerte).

- **Gólem** (Tierra): su «Coraza Espinosa» devuelve daño; es el ser *hecho para algo*. Pregunta: ¿puede elegir lo que fue hecho para ser? (Pico della Mirandola frente al ser humano «escultor de sí mismo»).
- **Céfiro** (Aire): libertad radical de Sartre; su «Viento a Favor» recuerda que nadie elige en el vacío (facticidad).
- **Huracán** (Aire): el absurdo de Camus y la revuelta; «Ojo de la Tormenta» = lucidez en el caos.
- **Brisa** (Aire): el asombro de Aristóteles y la voluntad de sentido de Frankl; «Soplo Sanador» = el sentido aparece al ocuparse de alguien.

Honestidad académica: los textos son una **lectura propia del juego** a partir de esos autores. Para defender esta parte, **lee al menos un fragmento original** de uno o dos autores (por ejemplo Levinas o Sartre) y quédate con una cita que puedas explicar con tus palabras. El propio Códice avisa que se deben consultar las obras originales para citar.

10. El modo LAN (dos PCs)

Cómo se sincronizan los dos PCs (solo viajan las jugadas)



LA IDEA CLAVE: los dos PCs hacen exactamente lo mismo

mismos equipos (CONFIG) + **mismo escenario** (CONFIG) + **misma semilla** (el azar igual) + **mismas jugadas** (mismo orden)

= el MISMO combate en los dos PCs

No se envía la vida de nadie: solo la jugada. Se llama modelo determinista.

Sin hilos: un Timer revisa cada 150 ms si llegó algo (LeerTexto devuelve null si no hay nada).



Pantalla de conexión: anfitrión (izquierda) o invitado (derecha).

10.1 Qué es cada cosa

- **Red LAN:** red local (mismo Wi-Fi o cable).
- **IP:** la «dirección» de un PC en la red (por ejemplo `192.168.0.104`).
- **Puerto:** un «número de puerta» del PC (aquí 8888).
- **Socket / TCP:** la conexión por la que se envían datos. TCP garantiza que lleguen completos y en orden.
- **Cliente-servidor:** uno espera (anfitrión/servidor) y el otro se conecta (invitado/cliente).

10.2 Cómo se usa

1. Los dos abren el juego y escriben su nombre. Ambos pulsan **«PARTIDA LAN (2 JUGADORES)»**.
2. **PC 1 (anfitrión):** ve su IP en pantalla, pulsa **«CREAR PARTIDA»**. (Si Windows pregunta por el Firewall: permitir en red privada.)
3. **PC 2 (invitado):** escribe esa IP, deja el puerto 8888 y pulsa **«CONECTAR»**.
4. Cada uno arma su equipo (el anfitrión también elige el escenario).
5. Se intercambian los equipos y empieza el combate. El anfitrión juega primero.

Probar en un solo PC: abre el juego dos veces; uno crea la partida y el otro se conecta a `127.0.0.1`.

10.3 Cómo funciona por dentro

Clases:

- `RedLan`: abre el puerto (`TcpListener`) o se conecta (`TcpClient`), envía y recibe **líneas de texto**.
- `FrmLan`: pantalla para elegir anfitrión o invitado. Usa un `Timer` que pregunta «¿ya se conectó alguien?».
- `FrmEsperaLan`: ventanita que espera el equipo del otro jugador.
- `ConfigPartida`: el mensaje con el equipo (y el escenario y la semilla si es el anfitrión).
- `FrmMenu.PartidaLan()`: coordina todo el flujo.

Sin hilos: para que la ventana no se congele, **nunca se espera**. `RedLan.LeerTexto()` devuelve `null` si no ha llegado nada, y un `Timer` de 150 ms lo vuelve a preguntar.

Los mensajes (líneas de texto separadas por `|`):

```
CONFIG|Los Valientes|0,3,6|2|424242     ← el anfitrión: su equipo, el escenario y la semilla
CONFIG|Mi Equipo|10,4,8                 ← el invitado: su equipo
ACCION|Habilidad|1|2                    ← una jugada (tipo, número de habilidad, aliado)
RENDIRSE                                ← abandonar
```

La idea importante — sincronización: no se envía «cuánta vida tiene cada personaje». Se envía **solo la jugada**. Los dos PCs crean **el mismo combate** (mismos equipos, mismo escenario, **misma semilla**) y aplican **las mismas jugadas en el mismo orden**, así que el resultado es idéntico en los dos. Es lo que se llama modelo **determinista**.

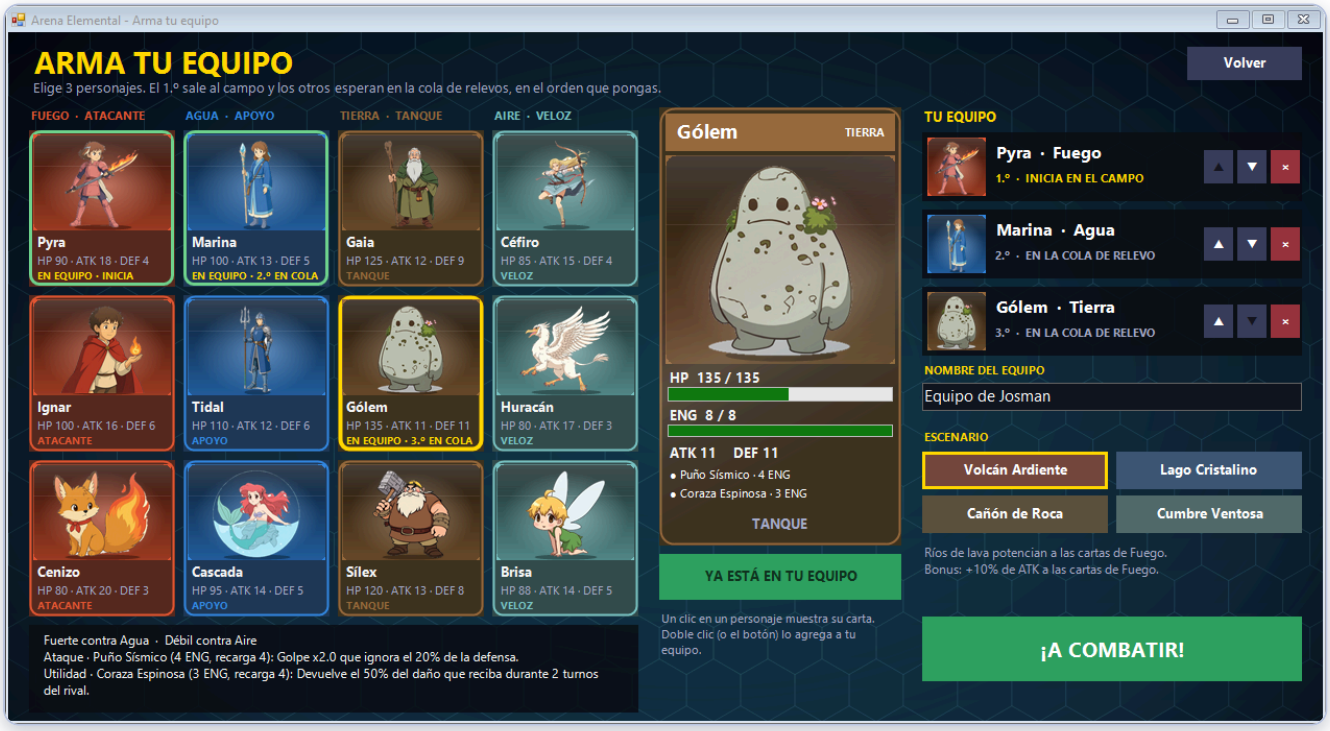
Por eso en `Combate` el azar (`Random`) se crea con una semilla: si fuera un `Random` distinto en cada PC, los Terremotos no coincidirían.

Quién es quién: en los dos PCs el equipo del anfitrión es el `Jugador` del `Combate` y el del invitado es el `Rival`. Cada `FrmCombate` usa las variables `local` (mi equipo) y `remoto` (el otro) para saber a quién mostrar a la izquierda.

Seguridad y errores que se manejan: IP mal escrita, puerto ocupado, anfitrión que no responde (10 s), el otro PC se desconecta o se rinde, mensajes inválidos (equipos repetidos, personajes inexistentes, jugadas imposibles: se ignoran o se corta la partida).

Limitación honesta: se probó con dos ventanas en un mismo PC (`127.0.0.1`) conectadas por sockets reales: ~70 jugadas hasta el final y estados idénticos. **No se ha probado con dos computadoras físicas.**

11. La interfaz explicada de forma sencilla



Pantalla de selección: galería de 12 personajes, vista previa y las 3 casillas del equipo.

11.1 Cómo se construye una ventana

Cada formulario hereda de `FormularioModerno` (que hereda de `Form`) y en su constructor crea los controles con `new` y los añade con `Controls.Add(...)`. Ejemplo mínimo:

```
Button btn = new Button();
btn.SetBounds(54, 394, 340, 50);          // x, y, ancho, alto
Estilo.EstilizarBoton(btn, "NUEVA PARTIDA (VS CPU)", Color.FromArgb(46, 160, 96));
btn.Click += BtnJugar_Click;
Controls.Add(btn);
```

11.2 `VistaCarta` (el control propio)

Un `UserControl` que dibuja un personaje como carta: cabecera, imagen (`PictureBox`), barras de vida y energía (`ProgressBar`), estadísticas y habilidades (`Label`). Se usa con **un solo método**:

```
carta.ActualizarDatos(personaje, "EN CAMPO", true);
```

Tiene dos tamaños (grande y pequeña) y un efecto «foil» holográfico (un degradado semitransparente en diagonal).

11.3 Las imágenes (`Ilustraciones.cs`)

Las 12 ilustraciones tienen fondo blanco. Para ponerlas sobre un color:

1. Se **reduce** la imagen a 340 px (más rápido).
2. Se mide la **claridad** de cada píxel (el menor de rojo, verde y azul).
3. **Relleno por inundación** (*flood fill*) con una `Queue<Point>`: desde los bordes se avanza por todos los píxeles casi blancos vecinos; eso es el fondo. (Es el mismo concepto de cola que en los relevos.)
4. Los blancos **pequeños** encerrados (ojos, barba) se respetan; los **grandes** (entre las piernas) también son fondo.
5. El fondo se pinta de **magenta** y se hace transparente con `Bitmap.MakeTransparent(Color.Magenta)`.
6. Se recorta al personaje y se dibuja sobre un degradado del color del elemento con `Graphics`.

Todo se guarda en **caché** (`Dictionary`) para no repetir el trabajo. Si una imagen falta o está dañada, la carta usa un color liso: **el programa no se cae**.

11.4 Efectos del combate

- **Fondo por escenario:** `FrmCombate.CargarFondoDelEscenario()` elige `volcan.jpg`, `lago.jpg`, `canon.jpg` o `cumbre.jpg` según `Escenario.ElementoAfin` (si no está, un color liso).
- **Números flotantes:** `AplicarAccion` compara la vida antes y después de cada jugada; un `Timer` hace subir una etiqueta (rojo = daño, verde = cura, naranja = Terremoto).
- **Banner del Terremoto** (`AvisoEvento`): una ventana sin borde con `Opacity` (translúcida). Se usó una ventana aparte porque un `Label` semitransparente no deja ver lo que tiene debajo.
- **Temblor:** un `Timer` mueve la ventana ±8 píxeles 12 veces.
- **Sonidos:** `System.Media.SoundPlayer`, con `try/catch` para que no pase nada si falta el `.wav`.
- **Banca en mano de cartas:** un `FlowLayoutPanel` horizontal con las cartas pequeñas; la primera (borde dorado) es la que entraría en el próximo relevo.

12. Preguntas que puede hacer el profesor (con respuesta)

1 ¿Qué es una clase abstracta y por qué `Personaje` lo es?

Una clase abstracta no se puede instanciar; sirve de molde. `Personaje` tiene todo lo común (estadísticas, daño, habilidades) pero no tiene sentido un personaje «sin elemento», así que solo se crean los de Fuego, Agua, Tierra y Aire. Además declara dos métodos `abstract` que cada hija **debe** implementar.

2 ¿Dónde hay herencia y polimorfismo?

`CartaFuego`, `CartaAgua`, `CartaTierra` y `CartaAire` heredan de `Personaje` y sobrescriben `ObtenerMultiplicador` (el triángulo) y `CrearCopia`. El motor llama `ObtenerMultiplicador` sobre un `Personaje` sin saber de qué elemento es: eso es polimorfismo.

3 ¿Qué es el encapsulamiento y dónde lo usaste?

Ocultar los datos y controlar cómo se cambian. La vida es un campo privado; la propiedad `Hp` limita el valor entre 0 y `HpMax`. Así ninguna otra clase puede dejar la vida en negativo.

4 ¿Por qué usaste una `Queue` y una `Stack` ?

La **cola** representa los relevos: el primero que espera es el primero que entra (FIFO) y el que sale se forma al final. La **pila** representa el historial: lo último que pasó es lo primero que se ve (LIFO).

5 ¿Qué diferencia hay entre `Peek` , `Dequeue` y `Pop` ?

`Peek` mira el elemento sin sacarlo. `Dequeue` saca el primero de la cola. `Pop` saca el último que se puso en la pila. (En el juego: `Peek` para ver quién entraría y la última jugada; `Dequeue` al relevar; `Push` / `Enqueue` para agregar.)

6 ¿Por qué hay copias de los personajes (`CrearCopia`)?

Para no modificar los personajes del catálogo. Si el combate bajara la vida del original, la siguiente partida empezaría con los personajes heridos.

7 ¿Qué hace el `Timer` ?

Ejecuta un método cada cierto tiempo sin congelar la ventana. Se usa para que la CPU «piense» 0.9 s, para el temblor, los números flotantes y para revisar la red.

8 ¿Cómo sabe la ventana que hiciste clic?

Por eventos: `btn.Click += BtnAtacar_Click;` registra el método que se ejecuta en el clic.

9 ¿Por qué la ventana no calcula el daño?

Por la separación de capas: las reglas están en `Modelos` / `Motor`, que no usan controles. La ventana solo muestra el estado. Así las reglas se pueden probar sin abrir ninguna ventana (así se verificó el proyecto).

10 ¿Cómo funciona la CPU?

Con reglas simples en orden de prioridad y un poco de azar (sección 5.8). No aprende; solo evalúa la situación (ventaja elemental, vida, energía, efectos activos).

11 ¿Cómo se relaciona el Terremoto con Cálculo 1?

El daño sale de la **derivada** de `f(t) = 2t² + 5t`, que es `f'(t) = 4t + 5`, acotada con un **límite** (`Min`) del 20 % de la vida (sección 8).

12 ¿Por qué `Min(f'(t), límite)`?

Porque `f'` crece sin límite y un Terremoto de 100+ de daño rompería el juego. El `Min` acota el valor y lo vuelve constante desde cierta ronda.

13 ¿Cómo funciona el modo LAN?

Sockets TCP: uno espera (`TcpListener`) y otro se conecta (`TcpClient`). Se envían solo las **jugadas**, y como los dos combates empiezan iguales (mismos equipos, escenario y **semilla**), quedan iguales siempre (sección 10).

14 ¿Por qué la semilla?

Un `Random` sin semilla da números distintos en cada PC. Con la misma semilla da la misma secuencia, así el Terremoto cae en las mismas rondas en los dos PCs.

15 ¿Qué pasa si se corta la red?

`RedLan` marca `Desconectado` y `FrmCombate` muestra un mensaje y cierra la partida registrándola como abandono. Si el otro se rinde, se muestra «ganas la partida».

16 ¿Por qué no usaste una base de datos?

Fue una decisión de alcance: el juego funciona 100 % en memoria con listas estáticas. Los scripts SQL (SQL Server y MySQL) quedaron en `BaseDatos\` para una versión futura. Las estadísticas se pierden al cerrar el programa.

17 ¿Qué es un enum y dónde lo usas?

Un conjunto de valores con nombre. `Elemento` (Fuego, Agua, Tierra, Aire), `EfectoHabilidad`, `TipoAccion`, `EstadoCombate`. Evitan usar números o textos «mágicos».

18 ¿Qué hace `out` en `PuedeEjecutar`?

Permite que el método devuelva **dos** cosas: un `bool` (¿se puede?) y un texto con el motivo si no se puede.

19 ¿Cómo manejas los errores?

Con validaciones que lanzan excepciones en las clases (nombre vacío, equipo con repetidos...) y con `try/catch` donde algo externo puede fallar (red, imágenes, sonidos). Una imagen o un sonido que falta **nunca** tumba el programa.

20 ¿Cómo quitas el fondo blanco de las imágenes?

Con un relleno por inundación que usa una `Queue<Point>`: parte de los bordes y avanza por los píxeles casi blancos vecinos; lo marcado se pinta de magenta y se vuelve transparente con `MakeTransparent`.

21 ¿Cómo está balanceado el juego?

Se simularon miles de combates de CPU contra CPU y se ajustaron las curaciones y habilidades hasta que los personajes ganaran más o menos entre el 44 % y el 60 % de las veces (Agua queda un poco por encima: es la facción de apoyo, con curaciones). Todos los combates terminan (máximo ~50 rondas gracias a la muerte súbita).

22 ¿Qué limitaciones tiene?

No se probó en un SharpDevelop 4.4 real (solo con el compilador C# 4 de .NET Framework), el LAN no se probó con dos computadoras físicas, las estadísticas no se guardan en disco y la CPU es de reglas simples.

23 ¿Qué mejorarías?

Guardar estadísticas en una base de datos, más personajes y escenarios, una CPU más estratégica, y modo de dos jugadores en el mismo PC.

24 ¿Qué aprendiste / qué fue lo más difícil?

(prepara tu respuesta personal): por ejemplo, entender cómo separar reglas e interfaz, o cómo sincronizar dos PCs con una semilla.

13. Ejercicios para practicar (cambios en vivo)

(No cambian la estructura del juego: son números y textos. Si agregas personajes nuevos, la pantalla de selección y el Códice están pensados para exactamente 12, así que eso no es un cambio pequeño.)

Si el profesor te pide modificar algo, estos cambios son pequeños y seguros. Haz cada uno una vez antes de la defensa.

1. **Cambiar el límite del Terremoto.** En `Combate.cs`, `LimiteTerremotoPct = 20` (pon 10 y explica que el `Min()` aplana el daño antes). Si además cambias la derivada en el método `DerivadaTerremoto` (por ejemplo `return 3 * t + 5;`, que corresponde a `f(t) = 1.5t² + 5t`), recuerda que los textos "f'(t) = 4t + 5" del historial, del banner y de la nota lateral están escritos a mano y habría que cambiarlos también.
2. **Cambiar la probabilidad del Terremoto.** `Combate.ProbabilidadTerremoto = 25` (pon 50 para ver más terremotos).
3. **Cambiar una estadística.** En `CatalogoBase.cs`, la línea de Pyra: cambia el 90 (vida) por 120.
4. **Cambiar una habilidad.** En `CatalogoBase.cs`, «Llamarada»: cambia el `2.0` (factor de daño) por `3.0`. Se ve en el daño y en el tooltip.
5. **Cambiar el bonus de un escenario.** `CatalogoBase.CargarEscenarios`: cambia `10` por `25`.
6. **Cambiar cuánta energía da «Pasar».** `Combate.EnergiaAlDescansar = 3`.
7. **Cambiar la energía por turno.** `Personaje.EnergiaPorTurno = 2` (la energía que recupera el personaje activo al empezar su turno).
8. **Cambiar un texto del Códice.** En `LorePersonajes.cs`, cambia el dilema de un personaje (el último texto de su bloque `Agregar(...)`) y míralo en el Códice. Sirve para explicar que el lore es un `Dictionary` que se busca por el nombre.

9. **Cambiar el color de un botón.** En `FrmMenu.cs`, `Estilo.EstilizarBoton(btnLan, "...", Color.FromArgb(196, 120, 24))`: cambia los tres números (rojo, verde, azul).
10. **Cambiar un texto.** Cualquier `Text` o `Label` en el formulario que quieras (por ejemplo, el subtítulo del menú).

Después de cada cambio: **F5** para ejecutar. Si algo se rompe, **Ctrl+Z**.

14. Cómo se verificó el proyecto

- **Compilación** con el compilador C# 4 de .NET Framework 4.0 (`csc.exe /langversion:4`), sin errores ni advertencias (x86 y AnyCPU). Esto garantiza que no se usó sintaxis posterior a C# 4.
- **Simulación de 4000 combates** CPU contra CPU con cola, habilidades y Terremoto: todos terminan, 0 errores de reglas (la cola nunca tiene derrotados, la vida y la energía están siempre en rango, el Terremoto nunca mata y nunca pasa del límite).
- **Prueba de equivalencia al simplificar el código:** antes de simplificar el backend se guardó la salida completa de 500 combates con jugadas y semillas fijas (más textos y descripciones). Tras simplificarlo, la salida fue **idéntica carácter por carácter**: las reglas no cambiaron.
- **Prueba LAN:** dos ventanas de combate conectadas por sockets reales en el mismo PC jugaron un combate completo con estados idénticos; también se probaron IP inválida, puerto ocupado, anfitrión inexistente y desconexión.
- **Revisión visual** de todas las pantallas generando imágenes de cada formulario.

No verificado: ejecución en un SharpDevelop 4.4 real, partida jugada a mano completa, modo LAN entre dos PCs físicos y el sonido (se comprobó que los `.wav` cargan, pero no se escucharon).

15. Sobre cómo se hizo el proyecto (transparencia)

Este proyecto se desarrolló con la ayuda de un asistente de programación con IA (Claude Code), que escribió y reorganizó mucho del código y sus pruebas. **Revisa el reglamento de tu materia y de tu universidad sobre el uso de herramientas de IA y declara su uso si lo exigen.** Para la defensa te conviene ser claro y honesto, y apoyarte en lo que sí puedes demostrar:

- **Lo que entiendes y puedes explicar:** esta guía. Léela con el código abierto hasta poder recorrer una jugada completa sin mirar (sección 6).
- **Tus decisiones de diseño:** el tema y los elementos, el triángulo de ventajas, los 12 personajes y sus habilidades, los escenarios, el balance, los textos filosóficos que elegiste, las imágenes y los fondos.
- **Lo que puedes modificar en vivo:** los ejercicios de la sección 13.

Si no puedes explicar una parte, es mejor decir «esta parte la voy a reforzar» que inventar una respuesta. Las partes más técnicas que quizá no domines aún están marcadas en la sección 16 como «avanzado»: puedes explicarlas en una frase general.

16. Qué es básico y qué es avanzado (para saber cuánto profundizar)

Básico — debes dominarlo: clases y objetos, propiedades, constructores, herencia, `abstract` / `override`, enums, listas, cola y pila, `if` / `for` / `foreach`, métodos, eventos `Click`, `Timer`, el recorrido de la sección 6, la fórmula del daño, el Terremoto con la derivada.

Intermedio — explícalo con tus palabras: `out`, constructores sobrecargados, miembros `static`, constructor estático, excepciones, `Dictionary`, `try/catch`, `Tag`.

Avanzado — basta una frase general:

- Dibujo con `Graphics` (degradados, esquinas redondeadas, el brillo «foil»): «dibujo formas y degradados por código con GDI+».
- Quitar el fondo de las imágenes (flood fill): «relleno desde los bordes con una cola».
- Sockets (`TcpListener` / `TcpClient`): «conexión TCP entre dos PCs; se envían líneas de texto».
- Sincronización por semilla: «los dos PCs ejecutan las mismas jugadas con el mismo azar sembrado».
- `Opacity` del banner y los números flotantes con `Timer`.

17. Glosario

- **Backend:** la parte del programa que no se ve (reglas, datos). Aquí: `Modelos`, `Motor`, `Datos`.
- **Frontend / interfaz:** lo que se ve (`Formularios`).
- **Clase:** el molde. **Objeto:** una cosa concreta hecha con el molde.
- **Propiedad:** forma de leer/escribir un dato controlando cómo.
- **Herencia:** una clase que reutiliza otra (`CartaFuego : Personaje`).
- **Polimorfismo:** objetos distintos que responden al mismo método de manera distinta.
- **Abstracta:** clase que no se puede instanciar directamente.
- **Cola (Queue):** primero en entrar, primero en salir (FIFO).
- **Pila (Stack):** último en entrar, primero en salir (LIFO).
- **Evento:** algo que ocurre (un clic). **Manejador:** el método que responde.
- **Timer:** reloj que ejecuta un método cada cierto tiempo.
- **Cooldown / recarga:** turnos que hay que esperar para volver a usar una habilidad.
- **Semilla:** número inicial de un generador de azar; la misma semilla da la misma secuencia.
- **Socket / TCP:** canal de comunicación entre dos programas por la red.
- **Determinista:** que con los mismos datos de entrada siempre da el mismo resultado.
- **Derivada:** la rapidez de cambio de una función. **Límite (aquí):** el tope que se le pone al daño con `Min`.
- **GDI+:** la parte de .NET para dibujar (`Graphics`).

★ Hoja de repaso (1 página)

El juego en 6 líneas

- Equipos de **3**; 1 en el campo y 2 en la **cola**.
- Por turno **1 acción**: Atacar · Habilidad (0 ataque / 1 utilidad) · Relevo · Pasar.
- Triángulo: **Agua > Fuego > Aire > Tierra > Agua** (×1.5 / ×0.5).
- Relevo = **FIFO**; no 2 seguidos; el derrotado no vuelve.
- Terremoto 25 % por ronda · Muerte súbita después de la ronda 30.

Fórmulas

- **Daño** = máx(1, ATK × (1+pot.) × factor × elemento × escenario − DEF × (1 − ignorada))
- **f(t)** = $2t^2 + 5t$ → **f'(t)** = $4t + 5$
- **Terremoto** = Min(f'(t), 20 % de HP máx.) · nunca deja menos de 1 HP
- **Muerte súbita**: ATK × (1 + 0.15 × (ronda − 30))
- **Curación** = HP máx × fracción + 2 × Empatía

POO: dónde está

- **Abstracta**: `Personaje`
- **Herencia**: `CartaFuego/Agua/Tierra/Aire : Personaje`
- **Polimorfismo**: `ObtenerMultiplicador`, `CrearCopia` (`override`)
- **Encapsulamiento**: campo `private hp` + propiedad `Hp` (0 a HpMax)
- **Enums**: Elemento, EfectoHabilidad, TipoAccion, EstadoCombate

Estructuras de datos

- **Queue** → `EquipoCombate.banca` (relevos)
- **Stack** → `Combate.historial` (jugadas)
- **List** → catálogo, miembros, partidas
- **Dictionary** → `LorePersonajes`
- **Push / Peek** (pila) · **Enqueue / Dequeue / Peek** (cola)

Las 3 capas

- **Modelos**: Personaje, Habilidad, Escenario
- **Datos**: CatalogoBase, RegistroPartidas, Lore
- **Motor**: Combate, EquipoCombate, IaRival, RedLan
- **Formularios**: ventanas (solo muestran y piden)
- Regla: «la ventana no calcula, el motor sí»

Una jugada, paso a paso

- Clic → `BtnAtacar_Click`
- `EjecutarJugador` → `AplicarAccion`
- `Combate.Ejecutar` → `Personaje.Atacar`
- `Registrar` (Push) → `ResolverDerrota` → `AvanzarTurno`
- `Refrescar()` dibuja y, si toca, el `Timer` llama a la CPU

LAN en una frase

- TCP: `TcpListener` (anfitrión) + `TcpClient` (invitado), puerto 8888.
- Se envían **solo jugadas** (texto con `|`).
- Misma **semilla** + mismas jugadas = mismo combate.
- Sin hilos: `Timer` de 150 ms.

Filosofía en una frase por elemento

- **Fuego**: voluntad de poder y finitud (Nietzsche)
- **Agua**: alteridad y cuidado (Levinas)
- **Tierra**: trascendencia y orden (Heidegger)
- **Aire**: libertad y sentido (Sartre, Frankl)